

ACTIONS · createActionGroup (NgRx 14+)

```
// actions/post.actions.ts
import { createActionGroup, props, emptyProps } from '@ngrx/store';

export const PostActions = createActionGroup({
  source: 'Post',
  events: {
    'Like Post': props<{ postId: string; prevLikes: number }>(),
    'Like Post Success': emptyProps(),
    'Like Post Failure': props<{ postId: string; prevLikes: number }>(),
  }
});

// Auto-generates: PostActions.likePost({ postId, prevLikes })
//                  PostActions.likePostSuccess()
//                  PostActions.likePostFailure({ postId, prevLikes })
```

REDUCER · pure function, immutable state

```
// reducers/post.reducer.ts
interface PostState {
  posts: Post[];
  loading: boolean;
  error: string | null;
}

const init: PostState = { posts: [], loading: false, error: null };

export const postReducer = createReducer(init,
// Optimistic update — fires BEFORE API call
on(PostActions.likePost, (state, { postId }) => ({
  ...state,
  posts: state.posts.map(p => p.id === postId
    ? { ...p, likes: p.likes + 1 } : p)
})),
// Rollback on failure
on(PostActions.likePostFailure, (state, { postId, prevLikes }) => ({
  ...state,
  posts: state.posts.map(p => p.id === postId
    ? { ...p, likes: prevLikes } : p)
})),
);
```

SELECTORS · memoized, compose-able

```
// selectors/post.selectors.ts
const selectPostState = createFeatureSelector<PostState>('posts');

export const selectAllPosts = createSelector(
  selectPostState, s => s.posts);

export const selectPostById = (id: string) =>
  createSelector(selectAllPosts,
    posts => posts.find(p => p.id === id));

// Derived data — recomputes only when posts change
export const selectLikedPosts = createSelector(
  selectAllPosts, posts => posts.filter(p => p.liked));
```

EFFECTS · side-effects, API calls

```
// effects/post.effects.ts
@Injectable()
export class PostEffects {
  likePost$ = createEffect(() =>
    this.actions$.pipe(
      ofType(PostActions.likePost),
      // mergeMap = allow concurrent likes
      mergeMap(({ postId, prevLikes }) =>
        this.api.likePost(postId).pipe(
          map(() => PostActions.likePostSuccess()),
          catchError(() => of(
            PostActions.likePostFailure({ postId, prevLikes })))
        )
      )
    );

  constructor(
    private actions$: Actions,
    private api: PostApiService
  ) {}
}
```

STORE REGISTRATION · wire it all up

```
// app.config.ts (standalone) or app.module.ts
provideStore({ posts: postReducer }),
provideEffects(PostEffects),
provideStoreDevtools({ maxAge: 25, logOnly: false }),

// Lazy feature (inside route providers):
provideState('posts', postReducer),
provideEffects(PostEffects),
```

COMPONENT · dispatch & select

```
// post-card.component.ts
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class PostCardComponent {
  @Input() postId!: string;

  post$ = toSignal(
    this.store.select(selectPostById(this.postId))
  );

  like() {
    const post = this.post$();
    if (!post) return;
    this.store.dispatch(
      PostActions.likePost({
        postId: post.id,
        prevLikes: post.likes // for rollback
      })
    );
  }
}
```

Which flattening operator?

switchMap	Load / Search	Cancels previous — use when only latest result matters
mergeMap	Like / Upload	Concurrent — use when all results matter
exhaustMap	Login / Submit	Ignores new while busy — prevents double-submit
concatMap	Queue / Order	Sequential — use when order must be preserved

Actions	Reducer	Selector	Effect	Dispatch
One action per user intent. Never reuse actions across features.	Always return new reference. Never state mutate directly.	createSelector() memoizes. Compose into small selectors bigger ones.	All side effects live here. Components call never API directly.	Dispatch actions, not state. Store is the of only source truth.